

PRACTICAL: 05

Aim: To find the maximum value of items that can be placed in a knapsack of capacity **W** without exceeding its weight limits, and to identify which items are selected, using Dynamic Programming.

Algorithm :

Algorithm

Input : Number of items n , knapsack capacity W , weight array w , and value array v .

Output : Maximum Possible value & the selected items.

- 1) Start
- 2) Read n, W , array w & array v .
- 3) Initialize base case:
 - i. set $dp[i][0] = 0$ for all i from 0 to n
 - ii. set $dp[0][j] = 0$ for all j from 0 to W
- 4) Repeat while $i \leq n$: (starting from $i=1$)
 - i. set $j = 1$
 - ii. Repeat while $j \leq W$:
 - * If $w[i] \leq j$, set $dp[i][j] = \max(v[i] + dp[i-1][j-w[i]], dp[i-1][j])$
 - * Else, set $dp[i][j] = dp[i-1][j]$
 - * Increment j
 - * Increment i
- 5) Display Dynamic Programming table dp & max value $dp[n][W]$
- 6) Find selected items:
 - i. set $j = W$
 - ii. Repeat while $j \geq 1$: (starting from $j = W$)
 - * If $dp[i][j] \neq dp[i-1][j]$, then display "Item i Taken" & set $j = j - w[i]$
 - * Else Display "Item i NOT taken"
 - * Decrement j
- 7) Stop.

Program / Code :

```
#include<iostream>
using namespace std;
class Knapsack {
private:
    int w[10];    // weights
    int v[10];    // values
    int n;        // number of items
    int W;        // capacity
    int dp[10][100]; // dp table
public:
    // — Get Input —
    void getInput() {
        cout << "Enter number of items  : ";
        cin  >> n;
        cout << "Enter knapsack capacity : ";
        cin  >> W;
        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin  >> w[i] >> v[i];
        }
    }
    // — Build dp table —
    void solve() {
        // base case — row 0 and col 0
        for(int i = 0; i <= n; i++) dp[i][0] = 0;
        for(int j = 0; j <= W; j++) dp[0][j] = 0;
        // fill table
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= W; j++) {
                if(w[i] <= j) {
```

```
// take or not take
dp[i][j] = max(
    v[i] + dp[i-1][j-w[i]],
    dp[i-1][j]
);
} else {
    // cannot take item i
    dp[i][j] = dp[i-1][j];
}
}
}
}
// — Print dp table —
void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "    ";
    for(int j = 0; j <= W; j++)
        cout << j << " ";
    cout << endl;
    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";
        cout << endl;
    }
}
// — Find selected items —
void findItems() {
    cout << "\nSelected Items:" << endl;
    int j = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
```

```
        cout << "Item " << i
            << " (w=" << w[i]
            << ", v=" << v[i]
            << ") TAKEN " << endl;

    j = j - w[i];
    } else {
        cout << "Item " << i
            << " (w=" << w[i]
            << ", v=" << v[i]
            << ") NOT taken " << endl;
    }
}
}
}
void showResult() {
    cout << "\nMaximum Value = "
        << dp[n][W] << endl;
}
};
int main() {
    Knapsack k;
    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();
    return 0;
}
```

Output :

```
C:\Users\NIKSHITH\DAA>g++ knapsack.cpp -o knapsack.exe

C:\Users\NIKSHITH\DAA>knapsack.exe
Enter number of items    : 3
Enter knapsack capacity : 7
Item 1 weight and value : 2
1
Item 2 weight and value : 5
6
Item 3 weight and value : 1
3

DP Table:
   0  1  2  3  4  5  6  7
i=0  0  0  0  0  0  0  0  0
i=1  0  0  1  1  1  1  1  1
i=2  0  0  1  1  1  6  6  7
i=3  0  3  3  4  4  6  9  9

Maximum Value = 9

Selected Items:
Item 3 (w=1, v=3) TAKEN
Item 2 (w=5, v=6) TAKEN
Item 1 (w=2, v=1) NOT taken

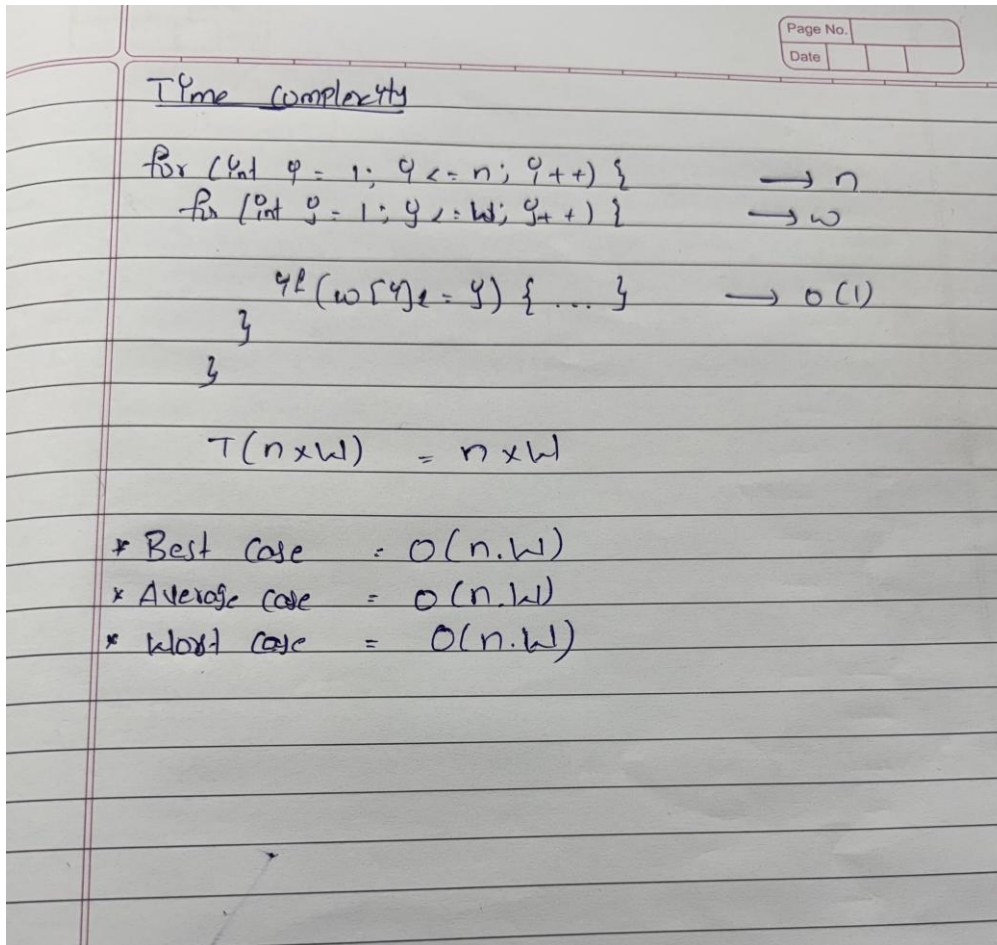
C:\Users\NIKSHITH\DAA>
```

Time Complexity :

Best Case : $O(n.W)$

Average Case : $O(n.W)$

Worst Case : $O(n.W)$



Conclusion :

The 0/1 Knapsack problem was successfully solved using Dynamic Programming. The algorithm guarantees the absolute maximum profit by systematically filling a storage table. It operates with a fixed time complexity of $O(n \cdot W)$ for all cases, as the program must always check every option to find the best match.